

# Shared Memory

## Part 1

Sistemas de Computadores  
Departamento de Engenharia Informática  
Instituto Superior de Engenharia do Porto

Luís Nogueira ([lmn@isep.ipp.pt](mailto:lmn@isep.ipp.pt))

# Shared memory

## ■ Processes operate in isolation

- Each process has its own memory space, preventing direct access to another process's memory

## ■ This restriction can be inconvenient when processes need to exchange data with increased efficiency and flexibility

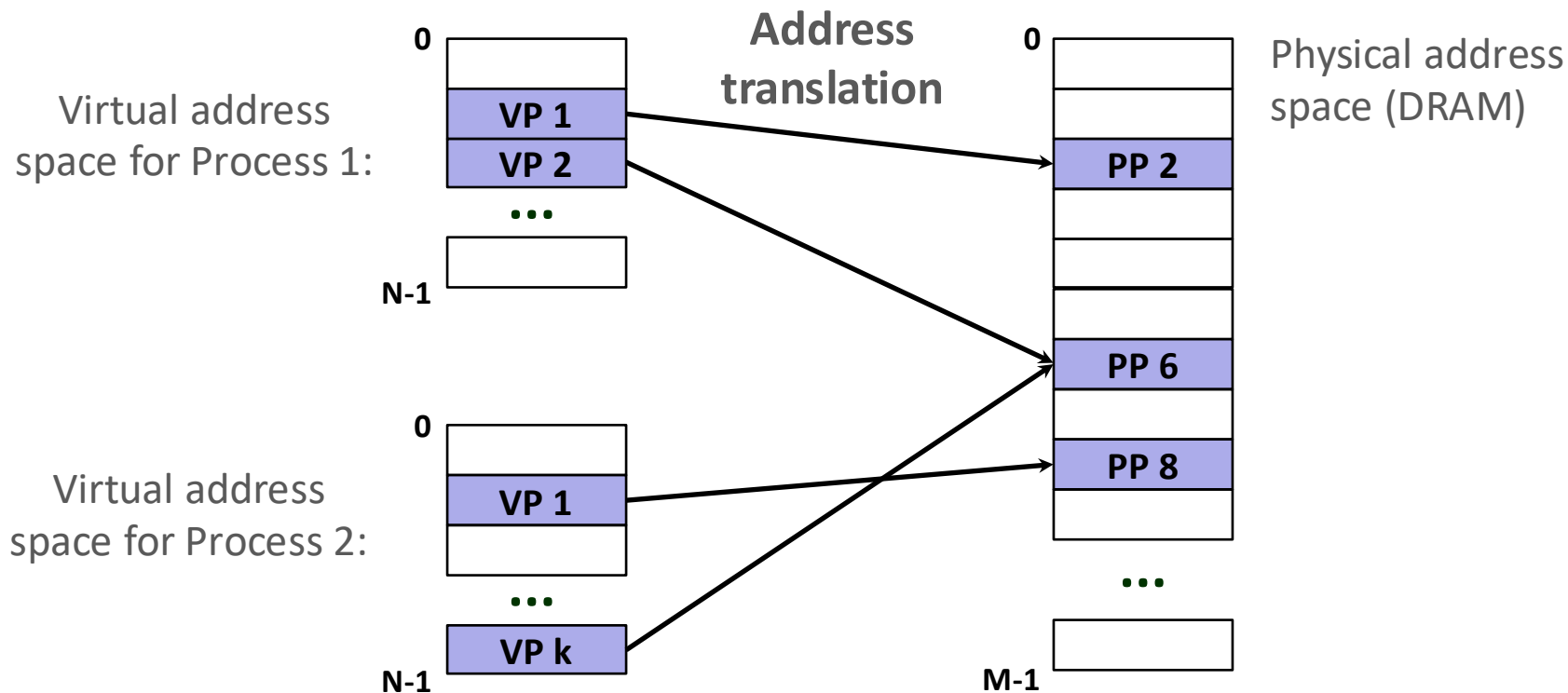
## ■ Shared memory as a solution

- A designated shared memory region allows multiple processes to access and modify the same data, enabling fast inter-process communication (IPC)

# Shared memory

## ■ Memory region which is **shared by several processes**

- Processes with or without a hierarchical link can use this region to communicate



# Advantages

## ■ Random access

- It is possible to directly access and change any part of the region

## ■ Low latency

- Accesses are direct (contrarily to pipes, for instance), without the intervention of the OS

## ■ High performance

- It is the fastest communication mechanism between processes

# Advantages

## ■ Scalability

- Suitable for high-throughput applications that require frequent inter-process communication

## ■ Flexibility

- Allows multiple processes to read and write simultaneously (with proper synchronization mechanisms)

# Disadvantages

## ■ Complex synchronization

- Requires mechanisms like semaphores or mutexes to prevent race conditions and data corruption

## ■ Debugging challenges

- Harder to trace errors due to direct memory manipulation by multiple processes

## ■ Memory management overhead

- Requires explicit allocation and deallocation, increasing complexity
- Implementation may vary across OSes, making cross-platform development more challenging

# POSIX shared memory

## ■ Implemented as temporary files

- On Linux, makes use of a dedicated `tmpfs` filesystem that is normally mounted under `/dev/shm`

## ■ Persistent

- Persists until `shm_unlink()` is called
- Closing (`close`) or unmapping (`munmap`) does NOT remove the object

## ■ A process may map a shared memory area (in POSIX referred frequently as an object), make changes, and disconnect

- These changes are visible for any process that maps this shared memory object

# The `/dev/shm` directory

- In Linux, the temporary file system (`tmpfs`) used for POSIX shared memory is mapped in `/dev/shm`
  - POSIX semaphores (discussed in future classes) are also mapped here
- Shared memory objects can be listed with `ls` and removed with `rm`, as files

```
$ ls /dev/shm
shmtest shm_ex1

$ rm /dev/shm/shmtest
```

# POSIX shared memory

## 1. Create/open

- `shm_open()`, `ftruncate()`, `mmap()`

## 2. Use the shared memory area

- With a pointer, similarly to any dynamic memory block

## 3. Remove (next class)

- `munmap()`, `close()`, `shm_unlink()`
- Creating shared memory is straightforward; ensuring its correct cleanup and removal is essential — and will be the topic of the next class

# POSIX shared memory

## ■ Usual headers:

- `<fcntl.h>` - File control options (e.g., `O_CREAT`, `O_RDWR`)
- `<sys/types.h>` - Types used by system APIs
- `<sys/stat.h>` - Constants for file permissions and opening modes
- `<sys/mman.h>` - Declarations for shared memory management (e.g., `shm_*` and `mmap()`)

## ■ Functions related to shared memory are part of the POSIX real-time library (`librt`)

- On some systems you must link with `-lrt` (older `glibc` versions)
- **Modern `glibc` integrates former `librt` functionality into `libc`, so explicit `-lrt` linking is no longer necessary**

```
$ gcc -Wall shm_test.c -o shm_test -lrt
```

# Create/open a shared memory area

## 1. Use the `shm_open()` function

- Creates and opens (or if already exists, just opens) a shared memory area
- Returns a file descriptor to be used in subsequent functions

## 2. Use the `ftruncate()` function

- Defines the size of the shared memory area

## 3. Use the `mmap()` function

- Maps the shared area into the process' address space

# The `shm_open` function

```
#include <sys/mman.h>
#include <sys/stat.h>      /* For constants "mode" */
#include <fcntl.h>         /* For constants O_* */

int shm_open(const char *name, int oflag, mode_t mode);
```

- **Creates and opens a new shared memory area, or just opens an existing one**
- **Returns**
  - A file descriptor to the shared memory
  - -1, in case of error

# The `shm_open` function

```
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>

int shm_open(const char *name, int oflag, mode_t mode);
```

- **The `name` parameter is the name to be used to identify the shared memory area**
  - Must start with `'/'`
  - On Linux, a new file is created under `/dev/shm`

# The `shm_open` function

```
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>

int shm_open(const char *name, int oflag, mode_t mode);
```

## ■ Creation flags

- `O_CREAT`: creates, if it not exists, or use existing one
- `O_EXCL`: (with `O_CREAT`) just creates, error if already exists
- `O_TRUNC`: if the shared memory object already exists, truncate it to zero bytes

## ■ Access flags

- `O_RDONLY`: open for reading only
- `O_RDWR`: open for reading and writing (no flag to just write)

# The `shm_open` function

```
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>

int shm_open(const char *name, int oflag, mode_t mode);
```

- **The mode parameter specifies the permissions of the shared memory object only when it is created**
  - If the object already exists, this parameter is ignored
  - The mode parameter is only relevant if `O_CREAT` is used
- **RWX for user/group/others**
  - Common value for you is `S_IRUSR | S_IWUSR`
  - Should be 0 (zero) if opening an existing area, since it has no effect

# The `ftruncate` function

```
#include <unistd.h>
#include <sys/types.h>

int ftruncate(int fd, off_t length);
```

- **Defines the size of the shared memory area and initializes the new space to zero**
  - Must be used after `shm_open()`, as memory block is **created with zero size**
- **If the area already exists, only changes if new size is different**
  - If new size is greater than existing size, expands the area, filling the new space with zeros
  - If new size is less than existing size, shrinks the area, discarding extra data

# The `ftruncate` function

```
#include <unistd.h>
#include <sys/types.h>

int ftruncate(int fd, off_t length);
```

- The `fd` parameter is the file descriptor returned by `shm_open()`
- The `length` parameter is the size, in bytes, of the shared memory area
- Returns
  - 0, if successful
  - -1, in case of error

# The mmap function

```
#include <sys/mman.h>

void *mmap(void *addr, size_t length, int prot, int flags,
           int fd, off_t offset);
```

## ■ Maps a shared memory object into the process' address space

- In practice, you are required to handle arguments `length` and `prot`

## ■ Returns

- A pointer to the object
- `MAP_FAILED ((void *) -1)`, in case of error

# The `mmap` function

```
#include <sys/mman.h>

void *mmap(void *addr, size_t length, int prot, int flags,
           int fd, off_t offset);
```

- **The `addr` parameter allows to request the area to be mapped in a specific address**
  - `NULL` in the usual case to let the OS define the address
- **The `length` parameter is the required size to be used**
  - Must be less than or equal to the size defined in `ftruncate()`
  - OS usually rounds to a multiple of the size of a memory page

# The mmap function

```
#include <sys/mman.h>

void *mmap(void *addr, size_t length, int prot, int flags,
           int fd, off_t offset);
```

## ■ The **prot** parameter specifies the protection flags

- `PROT_READ`: just read
- `PROT_READ | PROT_WRITE`: read and write

## ■ Must be consistent with **shm\_open()**

- Do not use `O_RDONLY` in `shm_open()` and then `PROT_READ | PROT_WRITE` in `mmap()`

# The `mmap` function

```
#include <sys/mman.h>

void *mmap(void *addr, size_t length, int prot, int flags,
           int fd, off_t offset);
```

- **The `flags` parameter controls the behavior of `mmap()`**
  - When using shared memory, always specify `MAP_SHARED` to ensure that modifications are visible to other processes
- **`fd` is the file descriptor returned by `shm_open()`**
- **`offset` defines the starting point of the mapping within the memory area**
  - It is typically set to zero to map the entire memory region from the beginning
  - If not, it must always be a multiple of the memory page size

# Example – create and open

```
int fd;  
void * addr;  
  
fd = shm_open("/shmtest", O_CREAT|O_EXCL|O_RDWR,S_IRUSR|S_IWUSR);  
ftruncate (fd, 100);  
addr = mmap(NULL, 100, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
```

- 1. Creates (O\_CREAT) a shared memory area named "/shmtest" with read/write permissions (O\_RDWR)**
  - The user is granted read (S\_IRUSR) and write (S\_IWUSR) access
  - If the shared memory segment already exists, the function returns an error due to the use of the O\_EXCL flag, which ensures exclusive creation
- 2. Sets the shared memory area size to 100 bytes**

# Example – create and open

```
int fd;  
void * addr;  
  
fd = shm_open("/shmtest", O_CREAT|O_EXCL|O_RDWR, S_IRUSR|S_IWUSR);  
ftruncate (fd, 100);  
addr = mmap(NULL, 100, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
```

## 3. Maps the shared memory region at an address chosen by the OS (NULL), using the entire size (100)

- It grants read and write permissions (`PROT_READ | PROT_WRITE`), ensuring consistency with the open mode
- The memory is shared (`MAP_SHARED`), linked to the previously opened file descriptor (`fd`)
- The mapping starts from the beginning (`offset = 0`), covering the full allocated space

# How to use

## ■ Use as a normal dynamic memory block

- The pointer returned by `mmap ( )` can be used just like a pointer returned by `malloc ( )`
- It is common to define a structured data layout for the shared memory region

## ■ Note that concurrent access to shared data may result in data inconsistency

## ■ Maintaining data consistency **requires synchronization mechanisms** to ensure the orderly execution of cooperating processes

# Manual synchronization

## ■ Next example:

- Shared memory between unrelated processes (no parent–child relationship)
- Reading and writing within a structured data format

## ■ No synchronization in the code!

## ■ Demands manual synchronization, by manually executing the writer first

- Recall that once data is written to the shared memory, it remains accessible to any process that maps the region
- The reader can retrieve and interpret the written data even after the writer has finished its operation

# Example – writer

```
typedef struct{
    int var1;
    int var2;
}shared_data_type;

int main(){
    int fd, data_size = sizeof(shared_data_type);
    shared_data_type *shared_data;

    fd = shm_open("/shmtest", O_CREAT|O_EXCL|O_RDWR, S_IRUSR|S_IWUSR);
    ftruncate (fd, data_size);
    shared_data = (shared_data_type*)mmap(NULL, data_size,
        PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);

    /* write */
    shared_data->var1 = 100;
    shared_data->var2 = 200;
}
```

# Example – reader

```
typedef struct{
    int var1;
    int var2;
}shared_data_type;

int main(){
    int fd, data_size = sizeof(shared_data_type);
    shared_data_type *shared_data;

    /* shm_open without creation flags, only access flag */
    fd = shm_open("/shmtest", O_RDWR, 0);
    shared_data = (shared_data_type *)mmap(NULL, data_size,
        PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);

    /* read */
    printf("%d\n", shared_data->var1);
    printf("%d\n", shared_data->var2);
}
```

# Synchronization with active waiting

- **Next example with synchronization with **active waiting** (also called **busy waiting**)**
  - In this method, a process continuously checks a condition in a loop until it becomes true
  - Wastes CPU cycles and scales poorly
  - Not acceptable in general-purpose systems
  - Passive synchronization (semaphores) will be discussed in future classes
- **Processes are related**
  - Parent is the writer and child is the reader, illustrating how data can be passed between them using active waiting

# Example with synchronization

```
typedef struct {
    int var1;
    int var2;
    volatile int new_data;           /* used for synchronized read/write operations */
}shared_data_type;

int main(){
    int fd, data_size = sizeof(shared_data_type);
    shared_data_type *shared_data;

    fd = shm_open("/shmtest", O_CREAT|O_EXCL|O_RDWR, S_IRUSR|S_IWUSR);
    ftruncate (fd, data_size);
    shared_data = (shared_data_type *)mmap(NULL, data_size,
        PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);

    shared_data->new_data = 0;

    pid=fork();

    ...
}
```

# Example with synchronization

```
if (pid > 0){  
    /* writer */  
    shared_data->var1 = 100;  
    shared_data->var2 = 200;  
    shared_data->new_data = 1;  
  
    wait(NULL);  
}  
else{  
    /* reader */  
    while(!shared_data->new_data);  
    printf("%d\n", shared_data->var1);  
    printf("%d\n", shared_data->var2);  
}  
  
return 0;  
}
```

 **Synchronization**

# Limitations of this approach

## ■ `new_data` should be declared `volatile`

- Prevents the compiler from caching the value in a register
- Forces the loop to re-read memory

## ■ However, `volatile` is NOT synchronization

- Does not guarantee atomicity
- Does not prevent race conditions
- Compilers and CPU can still reorder memory operations

## ■ Shared memory enables communication, but we still need **proper synchronization mechanisms**

- Next module: semaphores

# Exercise 1

## ■ Create two unrelated processes

- Writer: writes a string in a shared memory block
- Reader: reads from the shared memory block and prints the message

## ■ Use manual synchronization

- Ensure the reader process starts only after the writer has completed its operation, preventing race conditions

## ■ If you execute the reader several times, would you be able to read the same string value?

## Exercise 2

### ■ Implement a “Number 5 Lottery” game:

- Writer (parent): Places random numbers in the interval  $[1,5]$  in shared memory
- Reader (child): waits for a new number (spinning in a flag) and prints the number
- Both finish when the number is 5

### ■ Must guarantee that writer waits for reader to finish processing a number before generating a new one

- Writer also spins in a flag